Node.js
Interview Questions

Event loop internals, auth (JWT/OAuth), scaling, streams/clustering, and backend system design

Compiled from real interview experiences (The future University, CREATE Bytes, Innovation Labs, qpiAI, Fotello) + curated additions

---

Topics covered: event loop & libuv · JWT/OAuth2.0/RBAC · file uploads & security · rate limiting · clustering & worker threads · streams · Docker/Nginx deployment · Prisma/MongoDB aggregation · system design & machine coding
Difficulty split: Easy → Medium → Hard, each with theory, practical, and system-design-style coding questions

Format: Every section is split into **Easy** → **Medium** → **Hard**, and spans theoretical concepts, output-based/tricky code, and practical / machine-coding style problems.

EASY LEVEL

Node.js Basics

1. What is Node.js? Why is it well suited for I/O-heavy applications despite being single-threaded?

2. What is npm/package.json used for? Difference between dependencies and devDependencies?

3. What is the difference between require() (CommonJS) and import (ES Modules) in Node.js?

4. What is middleware in Express.js? Give a simple example of a logging middleware.

Express Basics

5. How do you set up a basic CRUD (Create, Read, Update, Delete) API using Express.js?

6. What is the purpose of the req, res, and next parameters in an Express middleware function?

7. How do you handle route parameters and query parameters in Express?

8. What is CORS and why do browsers block cross-origin requests by default?

Auth Basics

9. What is the difference between an access token and a refresh token?

10. What is JWT (JSON Web Token)? What are its 3 parts?

11. What is hashing and salting? Why should passwords never be stored in plain text?

HTTP & Networking Basics

12. Compare HTTP vs HTTPS vs TCP vs UDP. Which protocol does WebSocket use underneath, and why?

13. What are common HTTP status codes (200, 201, 400, 401, 403, 404, 500) and when is each used?

MEDIUM LEVEL

Event Loop & Internals

1. Explain how Node.js's event loop works internally (timers, pending callbacks, poll, check, close callbacks phases).

2. What is process.nextTick() and how does it differ from setImmediate() and Promise microtasks in ordering?

3. Explain the difference between microtask queue and macrotask (callback) queue in the context of Node.js specifically (not just the browser).

4. Why is Node.js described as 'single-threaded' when it can still handle concurrent I/O? Explain libuv's role and the thread pool.

Auth in Depth

5. Explain JWT authentication end-to-end: how a token is issued, verified via middleware, and where it should be stored (cookie vs localStorage) with security trade-offs.

6. Explain the OAuth 2.0 flow in depth — what does Google return (access token, refresh token, id token), and what does each one represent?

7. Explain a complete forget-password flow using OTP + JWT: from OTP generation, verification, to issuing a password-reset token.

8. Design and explain Role-Based Access Control (RBAC) in a Node.js backend — schema design + middleware enforcement.

9. If a JWT token is leaked, how do you protect your system? Discuss token expiry, refresh token rotation, and blacklisting strategies.

Security

10. How do you prevent SQL Injection and NoSQL injection on the server side?

11. If thousands/millions of files are being uploaded and one might be malicious, how do you protect your system (file type validation, sandboxing, virus scanning, size limits)?

12. How would you design client-side + server-side handling for uploading millions of images without crashing the frontend or backend — discuss chunking, direct-to-cloud upload via presigned URLs, and queuing.

13. What is rate limiting? What time window/limit would you pick for a login endpoint vs a public read API, and why?

File Uploads & Middleware

14. Explain how file uploads work with multer on the backend, including storage engines (disk vs memory) and integrating with Cloudinary/S3.

15. Explain the full lifecycle of an HTTP request through Express — from client to server and back — including where middleware, routing, and error handlers fit.

16. Explain middleware chaining in depth in Node.js/Express, including error-handling middleware (4-arg signature).

Scaling Concepts

17. Explain horizontal scaling vs vertical scaling in the context of a Node.js application.

18. Explain REST APIs vs GraphQL APIs — trade-offs in over/under-fetching, caching, and complexity.

19. What is progressive rendering and how does it relate to backend response streaming?

20. Explain lazy loading and Suspense at a conceptual level as it relates to backend-driven code splitting/data fetching.

HARD LEVEL

Scalability & System Design

1. Explain how you would implement horizontal scaling for a Node.js app in practice (load balancer + multiple app instances + shared session/cache store).

2. Implement (conceptually, with code) a round-robin load balancing algorithm in front of multiple backend instances.

3. Explain WebHooks vs Server-Sent Events (SSE) vs Polling — implementation details and when each is the right choice.

4. How do you scale WebSocket connections horizontally across multiple Node.js instances (Redis adapter / pub-sub, sticky sessions)?

5. Explain Redis Pub/Sub and how it's used to synchronize events (e.g. chat messages, socket broadcasts) across multiple server instances.

6. How do you design a scalable REST API — discuss caching layers, pagination, rate limiting, and statelessness.

Node.js Internals — Advanced

7. Explain multithreading in Node.js: the difference between the main event loop thread, the libuv thread pool, worker_threads, and child_process.fork().

8. When would you use a Worker Thread vs a Child Process vs simply relying on the async I/O model?

9. Explain Node.js clustering with a practical real-life example (cluster module, master-worker model, load distribution across CPU cores).

10. Explain streams in Node.js (Readable, Writable, Duplex, Transform) with a practical example like piping a large file without loading it fully into memory.

11. Explain the EventEmitter pattern in Node.js and implement a simplified version of it from scratch.

12. What is backpressure in streams, and how do you handle it?

Docker & DevOps for Node

13. Explain, end-to-end with practical steps, how you would containerize a Node.js app with Docker and deploy it to AWS (Dockerfile, multi-stage builds, image size optimization, environment variables, health checks).

14. Explain the role of Nginx as a reverse proxy in front of a Node.js app — SSL termination, load balancing, and static file serving.

15. How would you set up zero-downtime deployments for a clustered Node.js application?

Machine Coding / Practical Backend Design

16. Implement pagination with filtering entirely on the backend (Node.js + MongoDB or SQL) — including total count, page, limit, and sort query params.

17. Design and implement (end-to-end, from scratch) a payment integration flow with Razorpay or Stripe — order creation, webhook verification, and idempotency handling.

18. Design a rate limiter middleware in Express (token bucket or sliding window) without external libraries.

19. Design a fraud-detection style backend job: given a stream/array of transactions, flag userIds with 3+ transactions within a 10-minute window with similar amounts (within 10%). Discuss how you'd do this efficiently at scale (windowed aggregation, not brute force).

20. Design a reliable multi-step image-processing pipeline (upload → preprocess → external API call → store & deliver) that processes tens of thousands of images/day, ensuring each image progresses as far as possible even if a later step fails (queues, retries, dead-letter queue, idempotent steps).

21. How would you automatically group hundreds of real-estate photos uploaded from a single shoot into 'brackets' of the same scene (same scene, different exposures, uploaded in quick succession)? Discuss

timestamp clustering + perceptual image hashing/similarity.

ORM / Database Layer

22. Explain Prisma ORM in depth: schema definition, migrations, and how you'd structure a schema.prisma file for a multi-model relational app.

23. How would you debug and fix a broken/complex Prisma schema (relation mismatches, missing indexes) written entirely in one file?

24. Explain the MongoDB aggregation pipeline in depth — stages like $match, $group, $lookup, $facet, $unwind — with a practical query example.

25. What does the $facet stage in a MongoDB aggregation pipeline do, and when would you use it?

■ *Runs multiple independent aggregation pipelines within a single stage, useful for pagination + total count in one query*